COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	Oct 2, 2026	**DATE SUBMITTED:**	Oct 2, 2026
NAME:	Antonio, Christine Mae P.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

ILO A

Code

```
#include <iostream>
// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
```

```
class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }
    int N; // number of nodes in the graph
```

```
24      public:
25
26      adjNode **head; //adjacency list as array of pointers
27      // Constructor
28      DiaGraph(graphEdge edges[], int n, int N) {
29
30          // allocate new node
31          head = new adjNode*[N]();
32          this->N = N;
33
34          // initialize head pointer for all vertices
35          for (int i = 0; i < N; ++i)
36          head[i] = nullptr;
37
38          // construct directed graph by adding edges to it
39          for (unsigned i = 0; i < n; i++) {
40              int start_ver = edges[i].start_ver;
41              int end_ver = edges[i].end_ver;
42              int weight = edges[i].weight;
43              // insert in the beginning
44              adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
45              // point head pointer to new node
46              head[start_ver] = newNode;
47          }
48      }
```

```
49      // Destructor
50      ~DiaGraph() {
51          for (int i = 0; i < N; i++)
52          delete[] head[i];
53          delete[] head;
54      }
55      };
56
57      // print all adjacent vertices of given vertex
58  void display_AdjList(adjNode* ptr, int i) {
59
60      while (ptr != nullptr) {
61          std::cout << "(" << i << ", " << ptr->val
62          << ", " << ptr->cost << ") ";
63          ptr = ptr->next;
64      }
65
66      std::cout << std::endl;
67      }
```

```
69      // graph implementation
70  int main() {
71      // graph edges array.
72      graphEdge edges[] = {
73      // (x, y, w) -> edge from x to y with weight w
74      {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
75      };
76
77      int N = 6; // Number of vertices in the graph
78      // calculate number of edges
79      int n = sizeof(edges)/sizeof(edges[0]);
80
81      // construct graph
82      DiaGraph diagraph(edges, n, N);
83      // print adjacency list representation of graph
84      std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex,weight):"<<std::endl;
85
86      for (int i = 0; i < N; i++) {
87          // display adjacent vertices of vertex i
88          display_AdjList(diagraph.head[i], i);
89      }
90      return 0;
91  }
```

Output

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ILO B: B.1

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
    return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const {
    return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);

        for (auto &e : edges) {
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;

    }
    return os;
}
```

```cpp
template <typename T>
class Graph {
    public:
        // Initialize the graph with N vertices
        Graph(size_t N) : V(N) {
        }
        // Return number of vertices in the graph
        auto vertices() const {
        return V;
        }
        // Return all edges in the graph
        auto &edges() const {
        return edge_list;
        }

    void add_edge(Edge<T> &&e) {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) edge_list.emplace_back(e);
        else
        std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

    private:
        size_t V; // Stores number of vertices in graph
        std::vector<Edge<T>> edge_list;

};
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end()) {
                stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges){
        for (auto &j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}
```

```cpp
template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);

    for (auto v : dfs_visit_order) std::cout << v << std::endl;
}

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

ILO B: B.2

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>


template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const {
    return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const {
    return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++)
    {
    os << i << ":\t";
    auto edges = G.outgoing_edges(i);
    for (auto &e : edges) os << "{" << e.dest << ": " << e.weight << "}, ";
    os << std::endl;
    }
}
```

```cpp
template <typename T>
class Graph {
    public:
        Graph(size_t N) : V(N) {}
        auto vertices() const {
        return V;
        }

        auto &edges() const {
        return edge_list;
        }

        void add_edge(Edge<T> &&e){
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) edge_list.emplace_back(e);
        else
        std::cerr << "Vertex out of bounds" << std::endl;
        }

        auto outgoing_edges(size_t v) const {
            std::vector<Edge<T>> edges_from_v;
            for (auto &e : edge_list) {
                if (e.src == v) {
                    edges_from_v.emplace_back(e);
                }
            }
            return edges_from_v;
        }

        template <typename U>
        friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

    private:
        size_t V;
        std::vector<Edge<T>> edge_list;

};
```

```cpp
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges){
        for (auto &j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }

    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex)) queue.push(e.dest);
        }
    }

    return visit_order;
}
```

```cpp
template <typename T>
void test_BFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order) std::cout << v << std::endl;
}
int main() {
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Output

```
1:  {2: 2}, {5: 3},
2:  {1: 2}, {5: 5}, {4: 1},
3:  {4: 2}, {7: 3},
4:  {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:  {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:  {4: 4}, {7: 4}, {8: 1},
7:  {3: 3}, {6: 4},
8:  {4: 5}, {5: 3}, {6: 1},
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
The Depth-First Search (DFS) algorithm would help the person visit different locations from a location, backtrack, then to another from the same location.
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

_____

_____

_____

3. In the performed code, what data structure is used to implement the Breadth First Search?
Queue was used in the Breadth First Search.
4. How many times can a node be visited in the BFS?
In BFS, a node can be visited only once. It traverses through level-to-level, so it doesn't require backtracking, unlike DFS.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.